Experiment 6: Implementation of Link State Routing Algorithm

Tool Command Language (TCL CODE)

```
set ns [new Simulator]

set tf [open Linkstate.tr w]
$ns trace-all $tf

set nf [open Linkstate.nam w]
$ns namtrace-all $nf

$ns rtproto LS

set n0 [$ns node]
set n1 [$ns node]
set n2 [$ns node]
set n3 [$ns node]
set n4 [$ns node]
set n5 [$ns node]

$ns duplex-link $n0 $n1 1Mb 10ms DropTail
$ns duplex-link $n1 $n2 1Mb 10ms DropTail
$ns duplex-link $n1 $n3 1Mb 10ms DropTail
$ns duplex-link $n2 $n3 1Mb 10ms DropTail
$ns duplex-link $n2 $n4 1Mb 10ms DropTail
$ns duplex-link $n3 $n4 1Mb 10ms DropTail
$ns duplex-link $n4 $n5 1Mb 10ms DropTail

$ns cost $n0 $n1 1
$ns cost $n1 $n2 1
$ns cost $n1 $n3 3
$ns cost $n2 $n3 1
$ns cost $n2 $n4 5
$ns cost $n3 $n4 1
$ns cost $n4 $n5 1

$ns cost $n1 $n0 1
$ns cost $n2 $n1 1
$ns cost $n3 $n1 3
$ns cost $n3 $n2 1
$ns cost $n4 $n2 5
$ns cost $n4 $n3 1
$ns cost $n5 $n4 1

set tcp0 [new Agent/TCP]
$ns attach-agent $n0 $tcp0

set ftp0 [new Application/FTP]
$ftp0 attach-agent $tcp0
$ftp0 set packetSize_ 500
$ftp0 set interval_ 0.0001

set sink5 [new Agent/TCPSink]
$ns attach-agent $n5 $sink5
```

```
$ns connect $tcp0 $sink5

proc finish { } {
global ns nf tf
$ns flush-trace
close $tf
close $nf
exec nam Linkstate.nam &
exit 0
}
$ns at 1 "$ftp0 start"
$ns at 5 "$ftp0 stop"
$ns at 7 "$ftp0 start"
$ns rtmodel-at 7.5 down $n2 $n3
#$ns at 7.5 "$ns rtproto LS"
#$ns rtproto LS
$ns rtmodel-at 12 up $n2 $n3
$ns at 14 "$ftp0 stop"
$ns at 16 "finish"
$ns run
```

AWK CODE (Aho, Weinberger, and Kernighan)

```
BEGIN {
count=0;
}
{
if($1=="d")
{
count++;
}
}
END {
printf("Number of drop packets %d\n",count);
}
```

7. Write a program for a HDLC frame to perform the following (i). Bit stuffing (ii). Character stuffing.

(i). Bit stuffing

```
# include<stdio.h>
# include<string.h>
void main()
{
int bit[10],stuff[50],final[100],flag[]={0,1,1,1,1,1,1,0},i,j,l,one=0,k;
printf("Enter the length of bit stream:");
scanf("%d",&l);
printf("Enter the bit stream:");
for(i=0;i<l;i++)
{
scanf("%d",&bit[i]);
}
j=0;
for(i=0;i<l;i++)
{
  if(bit[i]==1)
{
    one++;
    stuff[j]=1;
j++;
    if(one==5)
    {
      stuff[j]=0;
j++;
      one=0;
    }
  }
else
{
  stuff[j]=0;
j++;
  one=0;
}
}
for(i=0;i<8;i++)
{
 final[i]=flag[i];
}
for(k=0;k<j;i++,k++)
{
 final[i]=stuff[k];
}
for(k=0;k<8;k++,i++)
{
 final[i]=flag[k];
}
printf("The stuffed stream is:");
for(k=0;k<i;k++)
{
printf("%d",final[k]);
}
}
```

OUTPUT/RESULT:

Enter the length of bit stream:10

Enter the bit stream:1 1 1 1 1 1 1 1 1 1

The stuffed stream is:01111110111111011111001111110

Enter the length of bit stream:10

Enter the bit stream:1 0 1 0 1 0 1 0 1 0

The stuffed stream is:01111110101010101001111110

7. (ii). Character stuffing

```
# include<stdio.h>
# include<string.h>
# include<conio.h>
main()
{
int i,j=0,l;
char dle,stx,etx;
char str[40],start[5],end[5],final[60],find[50];
printf("Enter data link character:\n");
dle=getche();
printf("\n Enter start character:\n");
stx=getche();
printf("\n Enter end character:\n");
etx=getche();
start[0]=dle;
start[1]=stx;
start[2]='\0';
end[0]=dle;
end[1]=etx;
end[2]='\0';
printf("\n enter the string: \n");
gets(str);
l=strlen(str);
for(i=0;i<l;i++)
{
if(str[i]==dle || str[i]==stx || str[i]==etx)
{
 find[j]=str[i];
j++;
 }
find[j]=str[i];
j++;
    }
    find[j]='\0';
strcpy(final,start);
strcat(final,find);
strcat(final,end);
printf("\n The stuffed string is:\n");
    puts(final);
}
```

OUTPUT/RESULT:

```
Enter data link character:
e
Enter start character:
s
Enter end character:
d
Enter the string:
good day

The stuffed string is:
esgoodd ddayed

--------------------------------


Enter data link character:
e
Enter start character:
s
Enter end character:
d
Enter the string:
morning

The stuffed string is:
esmorninged
```

8. Write a program for distance vector algorithm to find suitable path for transmission (Bellman Ford's Algorithm).

```
#include<stdio.h>
struct node
{
   int dist[20];
   int from[20];
}rt[10];
int main()
{
   int costmat[20][20];
   int nodes,i,j,k,count=0;
printf("\nEnter the number of nodes : ");
scanf("%d",&nodes);//Enter the nodes
printf("\nEnter the cost matrix :\n");
   for(i=0;i<nodes;i++)
   {
      for(j=0;j<nodes;j++)
      {
scanf("%d",&costmat[i][j]);
costmat[i][i]=0;
         rt[i].dist[j]=costmat[i][j];//initialise the distance equal to cost matrix
         rt[i].from[j]=j;
      }
   }
      do
      {
         count=0;
         for(i=0;i<nodes;i++)//We choose arbitary vertex k and we calculate the direct distance from
the node i to k using the cost matrix
         //and add the distance from k to node j
         for(j=0;j<nodes;j++)
         for(k=0;k<nodes;k++)
            if(rt[i].dist[j]>costmat[i][k]+rt[k].dist[j])
            {//We calculate the minimum distance
               rt[i].dist[j]=rt[i].dist[k]+rt[k].dist[j];
               rt[i].from[j]=k;
               count++;
            }
}while(count!=0);
      for(i=0;i<nodes;i++)
      {
printf("\n\n For router %d\n",i+1);
         for(j=0;j<nodes;j++)
         {
printf("\t\nnode %d via %d Distance %d ",j+1,rt[i].from[j]+1,rt[i].dist[j]);
         }
      }
printf("\n\n");
getch();
}
```

OUTPUT/RESULT:

Enter the number of nodes : 3
Enter the cost matrix :
0 2 7
2 0 1
7 1 0

For router 1

node 1 via 1 Distance 0
node 2 via 2 Distance 2
node 3 via 2 Distance 3

For router 2

node 1 via 1 Distance 2
node 2 via 2 Distance 0
node 3 via 3 Distance 1

For router 3

node 1 via 2 Distance 3
node 2 via 2 Distance 1
node 3 via 3 Distance 0

9. Implement Dijkstra's algorithm to compute the shortest routing path.

```
# include<stdio.h>
# include<conio.h>
main()
{
    struct
    {
        int pred;
        int len;
        char lab;
    }nodeinfo[20];
int n,i,j,min,src,dst,inf=9999,net[20][20],k;
printf("\n Enter the number of nodes:");
scanf("%d",&n);
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
if(i<j)
{
    printf("\n Enter the cost of the link %d to %d:",i,j);
    scanf("%d",&net[i][j]);
    net[j][i]=net[i][j];
    net[i][i]=0;
    net[j][j]=0;
}
}
}
printf("\n Enter the source node:");
scanf("%d",&src);
printf("\n Enter destination node:");
scanf("%d",&dst);
for(i=1;i<=n;i++)
{
    nodeinfo[i].pred=0;
    nodeinfo[i].len=9999;
    nodeinfo[i].lab='t';
}
k=dst;
nodeinfo[dst].len=0;
nodeinfo[dst].lab='p';
do
{
    for(i=1;i<=n;i++)
    {
        if((net[k][i]!=0) && (nodeinfo[i].lab=='t'))
        {
            if(nodeinfo[k].len+net[k][i]<nodeinfo[i].len)
            {
            nodeinfo[i].pred=k;
            nodeinfo[i].len=nodeinfo[k].len+net[k][i];
```

```
            }
          }
        }
min=inf;
for(i=1;i<=n;i++)
{
  if((nodeinfo[i].lab=='t') && (nodeinfo[i].len<min))
  {
     min=nodeinfo[i].len;
     k=i;
  }
}
nodeinfo[k].lab='p';
}
while(k!=src);
printf("\n cost of path between src and dst is=%d",nodeinfo[src].len);
printf("\n Shortest path is:\n");
do
{
     printf("\n %d to %d \n",k,nodeinfo[k].pred);
     k=nodeinfo[k].pred;
}
while(k!=dst);
getch();
}
```

OUTPUT/RESULT:

```
Enter the number of nodes:5
Enter the cost of the link 1 to 2:7222
Enter the cost of the link 1 to 3:9999
Enter the cost of the link 1 to 4:9999
Enter the cost of the link 1 to 5:1
Enter the cost of the link 2 to 3:1
Enter the cost of the link 2 to 4:9999
Enter the cost of the link 2 to 5:8
Enter the cost of the link 3 to 4:2
Enter the cost of the link 3 to 5:9999
Enter the cost of the link 4 to 5:2
Enter the source node:2
Enter destination node:5
cost of path between src and dst is=5
Shortest path is:
 2 to 3
 3 to 4
 4 to 5
```

10. For the given data, use CRC-CCITT polynomial to obtain CRC code.
Verify the program for the cases
i. Without error
ii. With error

```
#include<stdio.h>
main()
{
        int frame[25],gen[10],temp[30],i,j,m,n,count=0;
        printf("Enter the length of the message frame:");
        scanf("%d",&n);
        printf("Enter the message bits:");
        for(i=0;i<n;i++)
        {
                scanf("%d",&frame[i]);
        }
        printf("Enter the length of generator polynomial:");
        scanf("%d",&m);
        printf("Enter polynomial bits:");
        for(i=0;i<m;i++)
        {
                scanf("%d",&gen[i]);
        }
        for(i=0;i<n;i++)
        {
                temp[i]=frame[i];
        }
        for(i=n;i<(m+n-1);i++)
        {
                temp[i]=0;
                frame[i]=0;
        }
        for(i=0;i<n;i++)
        {
                if(temp[i]==1)
                for(j=0;j<m;j++)
                {
                        temp[i+j]=temp[i+j]^gen[j];
                }
                else
                for(j=0;j<m;j++)
                {
                        temp[i+j]=temp[i+j]^0;
                }
        }
        printf("The transmitted frame is:");
        for(i=0;i<m+n-1;i++)
        {
                frame[i]=frame[i]^temp[i];
                printf("%d",frame[i]);
        }
        printf("\n Enter the received message:");
        for(i=0;i<m+n-1;i++)
        {
                scanf("%d",&temp[i]);
```

```
        }
        for(i=0;i<n;i++)
        {
                if(temp[i]==1)
                for(j=0;j<m;j++)
                {
                        temp[i+j]=temp[i+j]^gen[j];
                }
                else
                for(j=0;j<m;j++)
                {
                        temp[i+j]=temp[i+j]^0;
                }
        }
        for(i=0;i<m+n-1;i++)
        {
                if(temp[i]==0)
                count++;
        }
        if(count==(m+n-1))
        printf("\n The received message is correct");
        else
        printf("\n The received message has an error.");
}
```

OUTPUT/RESULT:

Enter the length of the message frame:10

Enter the message bits:1 1 1 1 1 1 1 1 1 1

Enter the length of generator polynomial:4

Enter polynomial bits:1 1 1 1

The transmitted frame is:1111111111110

Enter the received message:1 1 1 1 1 1 1 1 1 1 1 1 0

The received message is correct

Enter the length of the message frame:10

Enter the message bits:1 1 1 1 1 1 1 1 1 1

Enter the length of generator polynomial:4

Enter polynomial bits:1 1 1 1

The transmitted frame is:1111111111110

Enter the received message:1 1 1 1 1 1 1 1 1 0 1 1 0

The received message has an error.

11. Implementation of Stop and Wait Protocol and Sliding Window Protocol

(a) Stop and wait protocol

```
# include <stdio.h>
# include <stdlib.h>
main()
{
        int i,n,r,a;
        n=5;
        printf("the number of packets are %d\n",n);
        for(i=1;i<=n;i++)
        {
                printf("the packet sent is:%d\n",i);
                r=rand()%2;
                if(r==1)
                {
                        a=rand()%2;
                        if(a==1)
                        {
                                printf("ack no:%d\n",i+1);
                        }
                        else
                        {
                        printf("no ack no:%d\n",i+1);
                                i--;
                        }
                }
        else
        {
                printf("timeout, resend\n");
                i--;
        }
 }
}
```

OUTPUT/RESULT:

```
the number of packets is 5
the packet sent is:1
ack no:2
the packet sent is:2
timeout, resend
the packet sent is:2
timeout, resend
the packet sent is:2
no ack no:3
the packet sent is:2
timeout, resend
the packet sent is:2
timeout, resend
the packet sent is:2
timeout, resend
the packet sent is:2
timeout, resend
the packet sent is:2
ack no:3
the packet sent is:3
ack no:4
the packet sent is:4
ack no:5
the packet sent is:5
no ack no:6
the packet sent is:5
no ack no:6
the packet sent is:5
no ack no:6
the packet sent is:5
timeout, resend
the packet sent is:5
no ack no:6
the packet sent is:5
timeout, resend
the packet sent is:5
no ack no:6
the packet sent is:5
timeout, resend
the packet sent is:5
ack no:6
```

11. (b) Sliding window protocol with automatic repeat-request (ARQ) mechanism

```
# include <stdio.h>
#include<stdlib.h>
main()
{
        int windowsize, sent=0,ack,i;
        printf("enter window size\n");
        scanf("%d",&windowsize);
        while(1)
        {
                for(i=0;i<windowsize;i++)
                {
                printf("frame %d has been transmitted\n",sent+1);
                sent++;
                if(sent==windowsize)
                  break;
            }
                ack=rand()%windowsize+1;
                printf("last ack receiced=%d\n",ack+1);
                if(ack==windowsize)
                break;
                else
                sent=ack;

        }
}
```

OUTPUT/RESULT:

```
enter window size
5
frame 1 has been transmitted
frame 2 has been transmitted
frame 3 has been transmitted
frame 4 has been transmitted
frame 5 has been transmitted
last ack receiced=3
frame 3 has been transmitted
frame 4 has been transmitted
frame 5 has been transmitted
last ack receiced=4
frame 4 has been transmitted
frame 5 has been transmitted
last ack receiced=6
```

12. Write a program for congestion control using leaky bucket algorithm.

```
# include <stdio.h>
main()
{
int bcktsize,pkt[25],i,j,iter,rate,line,total=0;
printf("Enter the bucket size and rate (in Mb):");
scanf("%d",&bcktsize);
printf("Enter the bucket rate (in Mb):");
scanf("%d",&rate);
printf("Enter the number of lines:");
scanf("%d",&line);
printf("Enter input packet rate of %d lines:\n",line);
for(i=0;i<line;i++)
scanf("%d",&pkt[i]);
printf("enter number of itrerations\n");
scanf("%d",&iter);
for(i=0;i<iter;i++)
{
        printf("\n iteration %d\n",i+1);
        for(j=0;j<line;j++)
        {
        total+=pkt[j];
        if(total<=bcktsize)
        printf("\n input from line %d with rate %d is added to the bucket\n current bucket
        size in (Mb) is %d \n",j+1,pkt[j],total);
        else
        {
        total-=pkt[j];
        printf("\n input from line %d with rate %d is thrown out of the bucket\n current
        bucket size in (Mb) is %d \n",j+1,pkt[j],total);
                }
                }
               if(total<=rate)
               {
                 printf("packet sent to output line at %d",total);
                 total=0;
                 printf("current bucket size is %d",total);
               }
              else
              {
                 total-=rate;
                 printf("\n.....\n");
        printf("\n packet sent to output line at rate %d \n current bucket size in (Mb) is
        %d",rate,total);
                printf("\n.....\n");
        }
    }
}
```

OUTPUT/RESULT:

```
Enter the bucket size and rate (in Mb):30
Enter the bucket rate (in Mb):10
Enter the number of lines:3
Enter input packet rate of 3 lines:
5
20
15
enter number of itrerations
2

 iteration 1

 input from line 1 with rate 5 is added to the bucket
 current bucket   size in (Mb) is 5

 input from line 2 with rate 20 is added to the bucket
 current bucket   size in (Mb) is 25

 input from line 3 with rate 15 is thrown out of the bucket
 current bucket size in (Mb) is 25

.....

 packet sent to output line at rate 10
 current bucket size in (Mb) is 15
.....

 iteration 2

 input from line 1 with rate 5 is added to the bucket
 current bucket   size in (Mb) is 20

 input from line 2 with rate 20 is thrown out of the bucket
 current bucket size in (Mb) is 20

 input from line 3 with rate 15 is thrown out of the bucket
 current bucket size in (Mb) is 20

.....

 packet sent to output line at rate 10
 current bucket size in (Mb) is 10
.....
```